

JNCC: Just NC Compiler

A Pure AI Compiler for XC to RISC-V64 Assembly

Jianlong Li

Department of Computer Science

Abstract

Traditional compilers rely on multiple intermediate representations (IR) and hand-crafted optimization passes to translate source code to machine code. In this paper, we present JNCC (Just NC Compiler), a novel pure AI compiler that directly translates XC source code to RISC-V64 assembly using deep learning, without any C or IR middle stages in the model path. Our system employs a hybrid architecture combining a deterministic rule-based Oracle backend with a learned Mamba state space model backend, achieving 100% runtime correctness with Oracle and 70% correctness with the neural model on our validation set. We demonstrate that transformer-based models can learn the complete compilation pipeline, including lexical analysis, syntax parsing, optimization, and instruction selection, in an end-to-end manner. The system supports multiple inference backends including pure neural, hybrid (neural + validation), and IR modes, enabling flexible trade-offs between speed and accuracy.

1. Introduction

Compilation is a fundamental process in computer science that transforms high-level source code into machine-executable instructions. Traditional compilers like GCC and LLVM rely on multiple intermediate representations (IR) and carefully engineered optimization passes to achieve high-quality code generation. While these compilers have been highly successful, they require extensive manual design and domain expertise to implement each compilation stage.

Recent advances in deep learning have shown promising results in code generation tasks, including code completion, program synthesis, and code translation. However, applying neural networks to full compilation pipelines remains challenging due to the complexity of syntax, semantics, and optimization requirements.

Key Contributions:

1. We propose the first pure neural compiler that translates XC source code to RISC-V64 assembly without any C or IR middle stages in the model path.
2. We design a hybrid architecture combining deterministic Oracle rules with learned Mamba state space models, achieving both high correctness and flexibility.
3. We implement end-to-end runtime validation using QEMU-based RISC-V64 emulation, ensuring semantic correctness of generated code.
4. We release a complete toolchain including lexer, parser, Oracle backend, neural inference module, and multi-backend pipeline.

2. System Architecture

2.1 XC Language

XC is a simple imperative language designed for compiler research. It supports variables, functions, control flow (if/else, while, for), and user-defined structures. XC syntax is designed to be clean and easily parseable, making it ideal for neural compilation experiments.

XC Language Features:

- **Variables:** `$x = 10`, `$x: int = 20`
- **Constants:** `@PI = 3.14`
- **Functions:** `% add(a: int, b: int) -> int { ^ a + b }`
- **Conditionals:** `? (cond) { }, ?: { }, ?? (cond) { }`

- **Loops:** @ (cond) { }, ~i=0; i<10; i=i+1 { }
- **Print:** ! x
- **Structures:** & Point { }

Sample XC Program:

```
# { $x = 10 $y: int = 20 $sum = x + y $prod = x * y ? (x > y) { ! "x > y" } ?:  
{ ! "x <= y" } ~i = 0; i < 10; i = i + 1 { ! i } % add(a: int, b: int) -> int {  
^ a + b } ^ add(x, y) }
```

2.2 Compilation Pipeline

Comparison of Traditional vs JNCC Pipelines:

Traditional Pipeline	JNCC Pipeline
1. XC Source	1. XC Source
2. Frontend (Lexer/Parser)	
9. Assembly	2. JNCC
	3. RISC-V64 Assembly

JNCC Backend Modes:

- **Oracle:** Deterministic rule-based compilation using hand-crafted translation rules
- **Model:** Pure neural inference using fine-tuned Mamba model
- **Hybrid:** Neural inference followed by Oracle validation
- **IR:** Neural inference with intermediate representation checking

2.3 Oracle Backend

The Oracle backend implements a complete rule-based compiler that serves as the ground truth generator. It includes:

- **Lexer:** Tokenizes XC source code into token sequences
- **Parser:** Builds abstract syntax trees (AST) from tokens
- **Semantic Analyzer:** Performs type checking and scope analysis
- **Code Generator:** Translates AST to RISC-V64 assembly using pattern matching rules

2.4 Neural Backend

The neural backend uses a fine-tuned Mamba-130M state space model with LoRA adapters. The model is trained on synthetic XC ↔ ASM pairs generated by the Oracle backend.

3. Experimental Results

3.1 Model Performance

Metric	Value	Source
Oracle Runtime Correctness	100%	Validation set
Model Runtime Correctness	70%	Validation set
Runtime Match Rate	50%	Validation set
Mean Generation Time	4.36s/sample	Local GPU
Median Generation Time	4.16s/sample	Local GPU
Oracle Latency	~0.0016s	Rule-based
Model Latency	44.36s	Neural inference

3.2 Oracle vs Model Comparison

Metric	Oracle	Model	Ratio
Runtime Correctness	100%	70%	Gap: 30%
Mean Latency	~0.0016s	44.36s	Model ~27,725x slower

3.3 Current Validation Status

Component	Status
XC Semantic Logic Check	Passed
Oracle Assembly Structure Check	Passed
Generated Assembly (100% GNU assembler pass rate)	Passed (100%)
Runtime Equivalence (Oracle)	Passed (100%)
Runtime Equivalence (Model)	Passed (70%)

4. Future Work

We are exploring a two-layer dynamic compilation architecture that combines SSM speed with LLM accuracy. The proposed system includes:

- **Layer 1:** Fast SSM inference for initial assembly generation
- **Layer 2:** Large language model (GPT-4/Claude) for verification and correction
- **Dynamic Routing:** Automatic selection based on confidence scoring
- **Self-Evolution:** Feedback loop using validation failures for iterative improvement

Expected Outcomes:

- **Runtime Correctness:** 70% → 90%+ via LLM verification
- **Latency:** Maintain <100ms for 80% of simple cases via SSM direct output
- **Adaptability:** Support new architectures with minimal fine-tuning
- **Self-Tuning:** Automatic confidence threshold adjustment based on validation feedback

5. Conclusion

In this paper, we presented JNCC, a pure AI compiler that demonstrates the feasibility of end-to-end neural compilation. Our system successfully translates XC source code to RISC-V64 assembly without traditional compiler infrastructure, achieving 100% correctness with the Oracle backend and 70% with the neural model. We believe this work provides a foundation for next-generation AI-powered compilers that can learn complete compilation pipelines from data.

References

- [1] Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.
- [2] Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374.
- [3] Fried, D., et al. (2022). InCoder: A Generative Model for Code Infilling and Synthesis. arXiv:2204.05999.